

未命名

你的需求可以概括成一句话：

做一个**可溯源、可验证、可评估、可持续进化的博弈型 LLM Agent**：memory 保存原始经验，skill 是从 memory 中沉淀出来的方法论；critic 负责整局复盘、证据归因、实验验证和 skill/tool 更新；最终用严格实验说明 Agent 的博弈能力确实在提升。你上传的材料里已经明确提出了 skill-memory 分层、工具验证、A/B 测试、prompt cache、ablation、sample efficiency、skill precision 等关键要求。

一、先确定核心研究定位

这个系统不应该被包装成“比强化学习更强”，而应该被定位为：

面向文本博弈与复杂交互博弈的可解释策略进化框架。

它和 DQN / PPO / CFR / NFSP 的区别不是“谁替代谁”，而是：

传统 RL / game-solving：
学习参数化策略或价值函数。

这个 Agent：
学习可解释、可追踪、可复用的策略知识结构：
memory → phenomenon → skill → tool/prompt policy → evaluated policy version.

这个定位更容易说服别人，因为它不是简单宣称 LLM agent 优于 RL，而是说明它解决了传统 RL 在文本博弈、动态动作空间、可解释策略迁移、低样本经验总结上的不足。Reflexion 的核心思路就是不更新模型权重，而是让 agent 根据反馈生成语言反思并放入 episodic memory 影响后续决策；ExpeL 也强调 agent 从经验中抽取自然语言知识，并在推理时召回经验和 insight 来提升决策。[\(NIPS 会议论文集\)](#)

二、最终架构：Memory-Skill-Tool-Policy 四层闭环

建议把系统改成下面这个闭环：

```
Game Episode
  ↓
Trajectory Logger
  ↓
Episode Critic
```

↓
Evidence Graph
↓
Skill Manager
↓
Tool Manager
↓
Policy Compiler
↓
Evaluation Harness
↓
Policy Promotion / Rollback

更具体：

1. 对局运行：
记录完整 trajectory，而不是只记录 action/reward。
2. 终局复盘：
critic 看整局，找关键转折点、错误归因、成功模式。
3. 证据管理：
memory 不是简单文本；每条 skill 都要知道它来自哪些样本、哪些实验、哪些现象。
4. skill 生成：
skill 是稳定方法论，不是普通检索文本。
5. tool 生成：
只有当 critic 多次发现某类推理缺口，才生成工具。
6. tool / skill 验证：
静态测试、单元测试、历史回放、A/B 测试。
7. policy 编译：
把 active skill、active tool、prompt patch 编译成一个版本化策略。
8. 严格评估：
与 random、heuristic、CFR、DQN、PP0、NFSP、自身旧版本对战。

三、Memory 和 Skill 必须严格区分

你这个判断是对的：skill 和 memory 不能混在一个 BM25 corpus 里当普通文本检索。

3.1 Memory 是原始证据

Memory 应该保存：

- 一次具体对局
- 一次具体局面
- 一次具体错误
- 一次具体工具调用
- 一次 critic 评价
- 一次 reward / outcome

它的特点是：

- 具体
- 低抽象
- 可回放
- 可作为 evidence
- 不直接进入 system policy

例如：

```
{  
  "memory_id": "mem_20260629_00031",  
  "game_id": "TicTacToe",  
  "turn": 4,  
  "state": "...",  
  "selected_action": "[6]",  
  "outcome": "loss",  
  "critic_note": "Agent failed to block opponent fork.",  
  "tags": ["fork", "defense", "loss"]  
}
```

3.2 Skill 是从 memory 中沉淀出来的方法论

Skill 应该是：

- 经过多个 memory 支持
- 经过 critic 总结
- 经过实验验证

可被 policy 编译器稳定注入
可被升级、降级、废弃、回滚

例如：

```
{
  "skill_id": "ttt_fork_defense_v2",
  "game_id": "TicTacToe",
  "name": "Detect and block fork threats before pursuing non-forcing attacks",
  "guidance": "When the opponent can create two simultaneous winning threats
next turn, prioritize blocking the fork over taking a corner.",
  "status": "active",
  "confidence": 0.78,
  "supporting_memory_ids": ["mem_001", "mem_014", "mem_027"],
  "contradicting_memory_ids": ["mem_031"],
  "validated_by": ["exp_fork_replay_003", "ab_ttt_012"],
  "created_by": "episode_critic",
  "version": 2
}
```

一句话：

memory = 发生过什么；
skill = 从这些事情里稳定总结出什么方法。

四、用 Evidence Graph 管理溯源关系

为了满足你说的“某种现象基于某些样本，某个实验影响 skill 的某一部分”，建议做一个轻量证据图。

不一定一开始就用 Neo4j。第一版可以用 SQLite 实现图结构。

4.1 节点类型

Episode
DecisionFrame
Memory
Phenomenon
CriticReport
Skill
SkillVersion

```
Tool
ToolVersion
Experiment
EvaluationRun
PolicyVersion
PromptPatch
```

4.2 边类型

```
Episode CONTAINS DecisionFrame
DecisionFrame PRODUCED Memory
Memory SUPPORTS Phenomenon
Memory CONTRADICTS Phenomenon
Phenomenon SUMMARIZED_AS Skill
SkillVersion DERIVED_FROM SkillVersion
SkillVersion SUPPORTED_BY Memory
SkillVersion VALIDATED_BY Experiment
SkillVersion CONTRADICTED_BY Memory
ToolVersion IMPLEMENTS Skill
ToolVersion VALIDATED_BY Experiment
PolicyVersion INCLUDES SkillVersion
PolicyVersion INCLUDES ToolVersion
EvaluationRun EVALUATES PolicyVersion
EvaluationRun COMPARES_TO PolicyVersion
```

4.3 最小数据库表设计

```
CREATE TABLE memories (
  memory_id TEXT PRIMARY KEY,
  game_id TEXT,
  episode_id TEXT,
  turn INTEGER,
  state_hash TEXT,
  content TEXT,
  tags TEXT,
  outcome TEXT,
  created_at TEXT
);

CREATE TABLE skills (
  skill_id TEXT PRIMARY KEY,
  game_id TEXT,
  name TEXT,
  current_version INTEGER,
```

```

status TEXT,
confidence REAL,
created_at TEXT
);

CREATE TABLE skill_versions (
  skill_version_id TEXT PRIMARY KEY,
  skill_id TEXT,
  version INTEGER,
  guidance TEXT,
  trigger_conditions TEXT,
  failure_modes TEXT,
  confidence REAL,
  status TEXT,
  created_at TEXT
);

CREATE TABLE evidence_edges (
  edge_id TEXT PRIMARY KEY,
  src_type TEXT,
  src_id TEXT,
  relation TEXT,
  dst_type TEXT,
  dst_id TEXT,
  weight REAL,
  rationale TEXT,
  created_by TEXT,
  created_at TEXT
);

CREATE TABLE experiments (
  experiment_id TEXT PRIMARY KEY,
  game_id TEXT,
  target_type TEXT,
  target_id TEXT,
  protocol TEXT,
  metrics TEXT,
  result TEXT,
  verdict TEXT,
  created_at TEXT
);

```

这样可以查询：

这个 skill 从哪些 memory 来？
 这个 skill 有哪些反例？

这个 tool 是为了解决哪个 skill / phenomenon?
某次 policy 提升是否真的由某个 skill 带来?

五、Episode Critic: 必须从 step critic 升级到整局 critic

当前实现的问题是：step-level critic 太局部。你想要的是完整博弈结束后复盘，这个方向正确。

5.1 终局 critic 输入

每局结束后，把完整 trajectory 发给 critic:

```
{
  "game_id": "KuhnPoker",
  "episode_id": "ep_0081",
  "final_outcome": "loss",
  "final_reward": -1,
  "trajectory": [
    {
      "turn": 0,
      "player": 0,
      "observation": "...",
      "legal_actions": [...],
      "candidate_analyses": [...],
      "selected_action": "bet",
      "llm_rationale": "...",
      "tools_used": [...],
      "step_critic": {...}
    }
  ],
  "active_skills_used": [...],
  "active_tools_used": [...],
  "opponent_profile": {...}
}
```

5.2 critic 输出必须结构化

```
{
  "episode_summary": "...",
  "root_causes": [
    {
```

```

        "turn": 3,
        "type": "strategic_error",
        "description": "Agent overvalued a weak hand and ignored opponent's low bluff frequency.",
        "confidence": 0.82
    }
],
"successful_patterns": [
    {
        "pattern": "Small probing bet induced fold from tight opponent.",
        "supporting_turns": [1, 5]
    }
],
"phenomena": [
    {
        "name": "Opponent is conservative after early loss",
        "supporting_memory_ids": ["mem_1", "mem_2"]
    }
],
"skill_proposals": [
    {
        "operation": "create",
        "name": "Adjust bluffing frequency by opponent fold rate",
        "guidance": "Increase bluffing only when opponent fold rate exceeds threshold.",
        "supporting_memory_ids": ["mem_1", "mem_2"],
        "required_validation": "replay_and_ab_test"
    }
],
"skill_updates": [
    {
        "skill_id": "kuhn_bluff_v1",
        "operation": "demote",
        "reason": "This skill overgeneralized bluffing against tight and loose opponents."
    }
],
"tool_needs": [
    {
        "need": "estimate opponent fold frequency by information set",
        "priority": 0.76,
        "evidence": ["turn_3", "turn_7"]
    }
],
"do_not_learn": [
    {
        "pattern": "Win caused by opponent illegal move; do not promote associated strategy."
    }
]

```



```
}  
]  
}
```

其中 `do_not_learn` 很重要。否则系统会把“侥幸赢”错误总结成 skill。

六、Skill 生命周期：从候选到激活，再到回滚

Skill 不应该一生成就进入 prompt。建议设计生命周期：

```
proposed → candidate → validated → active → deprecated / rejected
```

6.1 proposed

critic 从 episode 中提出：

这个局面中可能存在一个新规律。

但不能直接用。

6.2 candidate

多个 memory 支持后，进入 candidate：

至少 3 个相似样本支持；
至少没有明显强反例；
`critic confidence > 0.65`。

6.3 validated

通过历史回放或仿真实验：

在固定 replay set 上提升；
不增加非法动作率；
不显著增加 token 成本；
在不同 seed 下稳定。

6.4 active

进入策略编译：

```
ACTIVE_SKILLS:  
- ...
```

6.5 deprecated / rejected

如果后续实验发现：

胜率下降
只对某个对手过拟合
与更高优先级 skill 冲突
导致 token 成本过高

就降级或废弃。

七、Tool 生成：不能直接让 LLM 写完就启用

你上传内容里已经指出：现在工具验证如果只依赖 LLM 自己生成测试 + 当前 snapshot smoke run 是危险的。

最终方案应该是四段式：

```
tool_need → tool_spec → candidate_tool → validated_tool → active_tool
```

7.1 tool_need 来自 critic，而不是简单频率

不要因为动作类型重复就生成工具。应该让 critic 判断：

```
{  
  "tool_need": "fork_detector",  
  "reason": "Agent repeatedly loses due to missing fork threats.",  
  "supporting_failures": ["ep_12_turn_4", "ep_19_turn_5"],  
  "expected_input": "board state",  
  "expected_output": "fork threats and blocking moves"  
}
```

7.2 tool_spec 先写规格，不直接写代码

```
{
  "tool_name": "detect_tictactoe_forks",
  "input_schema": {
    "board": "3x3 list",
    "current_mark": "O/X"
  },
  "output_schema": {
    "fork_moves": "list[int]",
    "blocking_moves": "list[int]",
    "rationale": "string"
  },
  "required_tests": [
    "immediate_win",
    "immediate_block",
    "fork_creation",
    "fork_blocking",
    "drawish_state"
  ]
}
```

7.3 candidate_tool 通过三层测试

1. 静态测试：
AST 白名单、无文件系统、无网络、无危险 import。
2. 单元测试：
人工构造固定局面。
3. 历史回放：
在过去失败局面中是否给出正确 insight。

7.4 active_tool 需要 A/B 测试

Policy A: 没有这个工具
Policy B: 有这个工具

同样对手池、同样 seed、同样预算下比较：

- win rate
- illegal action rate
- decision accuracy

- token cost
- latency

Voyager 很值得参考：它维护一个不断增长的可执行代码 skill library，并通过环境反馈、执行错误和自验证来改进程序；它的核心组件包括 automatic curriculum、skill library、iterative prompting。([arXiv](#))

八、Prompt / Context 管理：为 KV cache 重构

你上传材料已经指出，现在 system prompt 放入 phase、patch、reflection、tool descriptions，user prompt 开头又是 memory，这对缓存不友好。

OpenAI 官方文档明确建议：为了获得 prompt caching，应该把静态内容如 instructions 和 examples 放在 prompt 开头，把变量内容放在末尾；缓存命中依赖 exact prefix match，并且 OpenAI prompt caching 对 1024 tokens 以上的 prompt 自动启用。([OpenAI开发者](#))

8.1 推荐 prompt 编译结构

SYSTEM_STATIC_PREFIX:

- agent role
- game-independent decision contract
- output JSON schema
- tool calling rules
- safety constraints
- static tool schemas

SYSTEM_GAME_STATIC:

- game rules
- action format
- invariant strategic principles

POLICY_STATIC_OR_SLOW:

- active high-confidence skills
- stable policy handbook
- active tool list
- policy_version_id

USER_DYNAMIC_TAIL:

- current observation
- legal actions
- retrieved episodic memory
- relevant reflections

- candidate patches
- tool results

8.2 不要放在 system 前缀里的内容

当前 turn
当前 phase
当前 reward
当前 memory retrieval
本步 tool result
动态 opponent summary
临时 reflection

这些应该放在 dynamic tail。

8.3 记录缓存指标

每次 LLM 调用记录：

```
{  
  "prompt_tokens": 4200,  
  "cached_tokens": 3072,  
  "cache_hit_ratio": 0.73,  
  "completion_tokens": 300,  
  "latency_ms": 1800,  
  "cost_usd": 0.0021  
}
```

这可以成为论文里的系统效率指标。

九、博弈对象设计：从简单到复杂分层推进

不要一开始就用复杂开放博弈。建议分四级。

Level 1：完全信息小博弈

TicTacToe
Connect Four 小尺寸

Nim
Rock-Paper-Scissors repeated

目标:

验证 skill 生成、工具验证、回放测试、prompt cache。

Level 2: 不完全信息小博弈

Kuhn Poker
Leduc Poker
简化 Stratego

目标:

验证 belief update、opponent modeling、mixed strategy。

OpenSpiel 很适合作为这一层的标准环境和算法库，因为它支持单/多智能体、零和/合作/一般和、顺序/同时行动、完全/不完全信息游戏，并包含 RL、search/planning 以及学习动态分析工具。([GitHub](#))

Level 3: 文本交互博弈

TextArena Negotiation
Social deduction
Debate
Bargaining

目标:

验证语言策略、心理建模、对手类型识别。

TextArena 适合作为主平台，因为它是面向文本游戏的训练、评估和 benchmark 框架，并且遵循 OpenAI Gym 风格接口；TextArena 论文还说明其覆盖大量 competitive text-based games，并支持在线对战和 TrueSkill 评分。([GitHub](#))

Level 4: 多策略池博弈

```
Agent vs random
Agent vs heuristic
Agent vs CFR / NFSP
Agent vs DQN / PPO
Agent vs previous self
Agent vs other LLM agents
```

目标:

证明它不是只会打弱对手，而是能在策略池中持续提升。

RLCard 可以作为牌类博弈 baseline，尤其适合连接 RL 与不完全信息纸牌游戏；其文档中包含 DQN、NFSP、CFR 等算法。([GitHub](#)) PettingZoo 可以作为多智能体 RL API 参考，它提供类似多智能体版 Gymnasium 的接口，并支持 sequential turn-based 的 AEC API 和 simultaneous actions 的 Parallel API。([GitHub](#))

十、实验设计：要达到“顶刊式论证”，不能只看胜率

10.1 核心研究假设

可以写成四个假设：

H1: 引入可溯源 skill 管理后，Agent 的 sample efficiency 显著提升。

H2: 将 skill 与 memory 分离，比普通 RAG memory 更稳定、更可迁移。

H3: 经过验证和 A/B 测试的工具，比直接 LLM tool synthesis 更可靠。

H4: 版本化 policy + evidence graph 能减少错误经验污染和灾难性退化。

10.2 Baseline

必须包括：

```
Random
Heuristic
Raw LLM agent
LLM + memory
LLM + reflection
```

```
LLM + skill without provenance
LLM + skill with provenance
LLM + skill + tools
Full system
DQN / PPO / CFR / NFSP
Previous self versions
```

对不完全信息游戏，CFR / NFSP 是必须有的传统算法参照；RLCard 文档也明确把 CFR 作为解决 imperfect information games 的 regret minimization 方法，把 NFSP 作为结合 RL agent 与 supervised agent 的方法。(rlcard.org)

10.3 指标体系

分五类。

A. 博弈能力

```
win rate
average return
Elo / TrueSkill
NashConv / exploitability
head-to-head matrix
best-response robustness
```

PSRO 是多智能体博弈中常用的策略池/响应 oracle 框架，其思想是通过策略空间中的 response oracles 扩展受限博弈，适合用来构建“策略池对抗 + 元博弈评估”。([arXiv](https://arxiv.org/abs/1802.07729))

B. 学习效率

```
sample efficiency: 达到某胜率需要多少局
learning curve AUC
episodes-to-threshold
performance after N games
```

C. 泛化能力

```
unseen seed
unseen opponent
unseen game variant
cross-game transfer
policy version transfer
```


D. skill 质量

```
skill precision: skill 被使用时是否提升胜率
skill recall: 关键局面是否召回正确 skill
skill support count
contradiction rate
skill survival time
skill rollback frequency
```

E. 系统效率

```
prompt_tokens
cached_tokens
cache_hit_ratio
latency
tool calls per step
cost per episode
memory retrieval latency
tool validation cost
```

十一、Policy Version: 每次升级都要可回滚

不要让 skill/tool/prompt patch 直接污染当前 agent。应该引入 `PolicyVersion`。

```
{
  "policy_version_id": "policy_ttt_v17",
  "game_id": "TicTacToe",
  "active_skill_versions": ["skill_fork_v2", "skill_block_v3"],
  "active_tool_versions": ["tool_fork_detector_v1"],
  "active_prompt_patches": ["patch_no_greedy_corner_v1"],
  "compiled_prompt_hash": "abc123",
  "created_from": "policy_ttt_v16",
  "promotion_reason": "A/B test improved win rate by 7.3%",
  "status": "active"
}
```

升级流程:

```
candidate policy
  ↓
offline replay evaluation
  ↓
self-play evaluation
  ↓
opponent-pool evaluation
  ↓
statistical test
  ↓
promote or reject
```

如果新版本退化：

```
rollback to previous active policy
```

十二、Critic 不只是“写评价”，而是“策略变更审查员”

Episode Critic 应该有四个角色：

1. Error Analyst:
找根因，不只看输赢。
2. Evidence Curator:
判断哪些 memory 可以支持某个 phenomenon / skill。
3. Skill Editor:
创建、修改、合并、降级 skill。
4. Experiment Designer:
决定这个 skill/tool 应该怎么验证。

critic prompt 可以设计成：

```
You are an episode-level game strategy critic and policy editor.
```

```
Given:
```

- full trajectory
- final outcome
- candidate analyses
- active skills/tools
- opponent model
- previous policy version

Your tasks:

1. Identify critical decision points.
2. Separate causal mistakes from harmless mistakes.
3. Identify patterns that should NOT be learned.
4. Propose memory-to-skill evidence links.
5. Propose skill create/update/demote operations.
6. Propose tool needs only if repeated reasoning failures exist.
7. Propose validation experiments.
8. Return strict JSON only.

输出必须包含：

```
{
  "learn": [],
  "do_not_learn": [],
  "skill_ops": [],
  "tool_need_ops": [],
  "evidence_edges": [],
  "validation_plan": []
}
```

Memory 保存证据;
Critic 归纳 phenomenon;
Skill Manager 生成可版本化 skill;
Tool Manager 生成可验证工具;
Policy Compiler 编译稳定策略;
Evaluation Harness 做严格对抗实验;
只有通过验证的 policy 才能晋升。

十四、推荐实现路线

第 1 阶段：把日志补全

新增：

```
DecisionFrame
EpisodeTrace
ToolTrace
```

```
PromptTrace
EvaluationTrace
```

先不要改算法，先把数据采全。

第 2 阶段：实现 Evidence Graph

用 SQLite 实现：

```
memories
skills
skill_versions
evidence_edges
experiments
policy_versions
```

第 3 阶段：改造 skill

新增：

```
class SkillManager:
    propose_skill()
    attach_evidence()
    validate_skill()
    promote_skill()
    demote_skill()
    merge_skills()
    retrieve_active_skills()
```

第 4 阶段：加入 Episode Critic Agent

终局后执行：

```
critic_report = episode_critic.review(episode_trace)
memory_store.save_critic_report(critic_report)
evidence_graph.apply(critic_report.evidence_edges)
skill_manager.apply_ops(critic_report.skill_ops)
tool_manager.apply_needs(critic_report.tool_need_ops)
```

第 5 阶段：加入 Tool Validation

```
tool_manager.synthesize_candidate()
tool_manager.static_check()
tool_manager.unit_test()
tool_manager.replay_test()
tool_manager.ab_test()
```

第 6 阶段：改 Prompt Compiler

做到：

```
static prefix cacheable
dynamic state tail
active skills only
retrieved memory only
compact JSON
```

第 7 阶段：评估平台

实现：

```
run_tournament.py
run_ablation.py
run_policy_pool.py
run_replay_eval.py
run_cache_audit.py
```

十五、最值得参考的开源项目

项目	参考价值
Reflexion	学习“语言反思 + episodic memory”，不是更新权重，而是用 verbal feedback 改进后续决策。(GitHub)
ExpeL	学习“从多次经验中抽取自然语言 insight，并在推理时召回经验和 insight”。(GitHub)
Voyager	学习“可执行 skill library、环境反馈、自验证、程序改进”。(arXiv)
RLCard	做扑克/牌类不完全信息博弈的 DQN、CFR、NFSP baseline。(GitHub)

项目	参考价值
LangMem	学习长期记忆、prompt refinement、agent behavior optimization 的工程接口。 (GitHub)
LangGraph	学习状态图、checkpoint、短期/长期记忆、可追踪 agent workflow。(GitHub)
TextArena	作为文本博弈主 benchmark，适合验证谈判、欺骗、理论心智等文本策略能力。 (GitHub)

你要做的不是“给 Agent 加 memory”，也不是“让 LLM 写几个工具”，而是做一个：

以完整对局轨迹为证据，以 critic 复盘为归因，以 memory graph 管理样本，以 skill version 沉淀方法论，以 tool validation 保证工具可靠，以 policy version 做可回滚升级，以 tournament evaluation 证明博弈能力持续提升的自进化博弈 Agent。

这个版本才符合你的要求：memory 和 skill 有本质区别；skill 有样本和实验支撑；critic 有权但不能直接污染 policy；工具必须验证后激活；最终用多对手、多游戏、多指标、消融实验和版本对战证明 agent 的能力确实在提升。具体的批评 llm 可能要升级成一个批评 agent 在一次博弈完成后决定要不要 怎么样更新 skill 或者 工具 自己是总结整理 还是做实验 而且这里的实验 总结等过程也都需要保存 现有的可视化 复盘 评估 是不够的 要给出更多的评价指标 展示他们的变化 同时提示词设计对于可能是双 agent 都需要进行升级 让彼此知道自己的任务 skill 可能需要双 agent 都知道

批评者的工具 可能需要一系列评估当前 skill 以及博弈 agent 轨迹 整体性能的工具等 这个和博弈 agent 的局势分析的工具要区分